

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
—o0o—



BÁO CÁO KẾT QUẢ

HỌC PHẦN: PROJECT I

HỆ THỐNG NHẬN DẠNG NGÔN NGỮ KÝ HIỆU

NGUYỄN ĐÀO NAM HẢI
Hai.NDN235321@sis.hust.edu.vn

Lớp	: 755566
Giảng viên hướng dẫn	: Hoàng Việt Dũng
Sinh viên thực hiện	: Nguyễn Đào Nam Hải
Chương trình đào tạo	: Kỹ thuật máy tính

Hà Nội, 01/2026

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong kỷ nguyên của Cách mạng Công nghiệp 4.0, việc ứng dụng công nghệ để giải quyết các vấn đề xã hội và hỗ trợ người khuyết tật đang trở thành một xu hướng nhân văn quan trọng. Đối với cộng đồng người khiếm thính, ngôn ngữ ký hiệu (Sign Language) là phương tiện giao tiếp chính, nhưng rào cản ngôn ngữ đối với đại đa số người bình thường đã tạo ra những khó khăn không nhỏ trong việc hòa nhập cộng đồng, tiếp cận giáo dục và y tế. Nhận thức được tính cấp thiết của vấn đề, em đã lựa chọn đề tài **"Xây dựng hệ thống nhận diện ngôn ngữ ký hiệu thời gian thực sử dụng AI và ESP32"** cho học phần Project I.

Báo cáo này trình bày toàn bộ quy trình nghiên cứu, thiết kế và triển khai một hệ thống hỗ trợ giao tiếp hoàn chỉnh. Về mặt công nghệ, hệ thống áp dụng mô hình Học sâu (Deep Learning) với Mạng Nơ-ron Tích chập (CNN) để xử lý hình ảnh và nhận diện cử chỉ tay từ Camera máy tính. Kết quả nhận diện sau đó được truyền tải thông qua giao thức mạng đến vi điều khiển ESP32. Tại đây, một Web Server được xây dựng với giao diện người dùng hiện đại (Glassmorphism), cho phép hiển thị video trực tiếp và kết quả phiên dịch, hỗ trợ người dùng ghép nối các ký tự thành văn bản có ý nghĩa.

Quá trình thực hiện đồ án là một hành trình học tập và trải nghiệm thực tế quý báu. Em đã có cơ hội vận dụng và kết nối các kiến thức đa ngành: từ Thị giác máy tính (Computer Vision), Xử lý ảnh, Huấn luyện mô hình AI, đến Lập trình nhúng và Phát triển giao diện Web. Đồ án không chỉ dừng lại ở việc áp dụng lý thuyết mà còn đòi hỏi kỹ năng giải quyết các bài toán thực tế như tối ưu hóa độ trễ, xử lý đa luồng và thiết kế trải nghiệm người dùng (UI/UX).

Với kết quả đạt được là một hệ thống hoạt động ổn định, có khả năng nhận diện bảng chữ cái với độ chính xác cao trong thời gian thực, em hy vọng đồ án sẽ đóng góp một giải pháp công nghệ chi phí thấp, giàu tính thực tiễn. Đồng thời, đây sẽ là nền tảng vững chắc để em tiếp tục nghiên cứu các bài toán phức tạp hơn như nhận diện cử chỉ động hay triển khai mô hình AI trực tiếp trên các thiết bị biên (Edge AI) trong tương lai.

MỤC LỤC

1	GIỚI THIỆU ĐỀ TÀI	1
1.1	Đặt vấn đề	1
1.2	Mục tiêu và phạm vi đề tài	1
1.2.1	Mục tiêu đề tài	1
1.2.2	Phạm vi đề tài	1
1.3	Định hướng giải pháp	2
1.4	Bố cục đồ án	2
2	CƠ SỞ LÝ THUYẾT	3
2.1	Tổng quan về Học sâu và Thị giác máy tính	3
2.1.1	Thị giác máy tính (Computer Vision)	3
2.1.2	Học sâu (Deep Learning)	3
2.2	Tiền xử lý ảnh và Biểu diễn dữ liệu	3
2.2.1	Kỹ thuật Cắt vùng quan tâm (ROI)	3
2.2.2	Chuyển đổi không gian màu (Grayscale)	4
2.2.3	Chuẩn hóa dữ liệu (Normalization)	4
2.3	Mạng Nơ-ron Tích chập (CNN)	4
2.3.1	Các thành phần cốt lõi	4
2.3.2	Hàm mất mát và Tối ưu hóa	5
2.4	Hệ thống nhúng và Giao thức truyền thông	5
2.4.1	Vi điều khiển ESP32	5
2.4.2	Giao thức truyền tải video MJPEG	5
2.5	Các độ đo đánh giá hiệu suất	6
2.5.1	Độ chính xác (Accuracy)	6
2.5.2	Precision, Recall và F1-Score	6
3	QUY TRÌNH HOẠT ĐỘNG CỦA HỆ THỐNG	7
3.1	Tổng quan Quy trình Hoạt động	7
3.2	Thu thập và Chuẩn bị Dữ liệu	7
3.3	Tiền xử lý Dữ liệu (Image Preprocessing)	8
3.4	Trích xuất Đặc trưng (Feature Extraction)	8

3.5	Huấn luyện mô hình	8
3.6	Quy trình Tích hợp và Triển khai	9
3.7	Đánh giá mô hình	10
4	CÁC THƯ VIỆN VÀ CÔNG NGHỆ SỬ DỤNG	11
4.1	Môi trường phát triển	11
4.2	Các thư viện Học sâu và Xử lý ảnh	11
4.2.1	TensorFlow và Keras	11
4.2.2	OpenCV (Open Source Computer Vision Library)	11
4.2.3	NumPy	12
4.3	Các công nghệ Backend và Server	12
4.3.1	Flask	12
4.3.2	Thư viện Threading	12
4.4	Công nghệ Frontend (Giao diện người dùng)	12
4.5	Công nghệ Lập trình Nhúng cho ESP32	13
4.5.1	Thư viện WiFi.h	13
4.5.2	Thư viện WebServer.h	13
5	KẾT QUẢ THỰC NGHIỆM	14
5.1	Phân tích Dữ liệu khám phá (EDA)	14
5.1.1	Thông kê dữ liệu	14
5.1.2	Đánh giá sự cân bằng dữ liệu	14
5.2	Kết quả huấn luyện mô hình	14
5.2.1	Biểu đồ Huấn luyện (Training History)	14
5.2.2	Ma trận nhầm lẫn (Confusion Matrix)	16
5.3	Kết quả thực nghiệm trên hệ thống	16
5.3.1	Giao diện người dùng (Web Interface)	16
5.3.2	Khả năng nhận diện thời gian thực	17
5.4	Đánh giá hiệu năng hệ thống	17
6	VẤN ĐỀ VÀ GIẢI PHÁP	18
6.1	Các vấn đề và khó khăn gặp phải	18
6.1.1	Sự nhầm lẫn giữa các ký tự có hình dạng tương tự	18
6.1.2	Ảnh hưởng của điều kiện môi trường	18
6.1.3	Độ trễ truyền dẫn (Latency)	18

6.2	Các giải pháp đã áp dụng	18
6.2.1	Tăng cường dữ liệu (Data Augmentation)	19
6.2.2	Tinh chỉnh ngưỡng tin cậy (Confidence Threshold)	19
6.3	Kỹ thuật Xử lý Đa luồng (Multi-threading)	19
6.3.1	Cơ chế hoạt động	19
6.3.2	Hiệu quả đạt được	20
7	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	21
7.1	Kết luận	21
7.2	Hạn chế của đề tài	21
7.3	Hướng phát triển trong tương lai	21
7.3.1	Triển khai Edge AI (AI tại biên)	22
7.3.2	Nhận diện cử chỉ động (Dynamic Gestures)	22
7.3.3	Tích hợp Text-to-Speech (Chuyển văn bản thành giọng nói)	22
	TÀI LIỆU THAM KHẢO	23

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Ngôn ngữ ký hiệu (Sign Language) là phương tiện giao tiếp chính của cộng đồng người khiếm thính. Tuy nhiên, sự khác biệt về ngôn ngữ tạo ra rào cản lớn khiến họ khó hòa nhập với cộng đồng, cũng như hạn chế trong việc tiếp cận các dịch vụ xã hội.

Các giải pháp hỗ trợ hiện nay như thuê phiên dịch viên thường tốn kém và không chủ động. Các thiết bị phần cứng chuyên dụng (găng tay cảm biến) lại cồng kềnh và chi phí cao. Trong khi đó, công nghệ Thị giác máy tính (Computer Vision) và Internet vạn vật (IoT) đang phát triển mạnh mẽ, cho phép xử lý hình ảnh phức tạp với chi phí thấp.

Xuất phát từ thực tế đó, đề tài tập trung xây dựng một hệ thống nhận diện ngôn ngữ ký hiệu tự động, sử dụng Camera máy tính để nhận diện và hiển thị kết quả lên giao diện Web của vi điều khiển ESP32. Đây là giải pháp cân bằng giữa hiệu năng và chi phí, mang tính ứng dụng cao.

1.2 Mục tiêu và phạm vi đề tài

1.2.1 Mục tiêu đề tài

Mục tiêu cốt lõi là xây dựng một hệ thống hoàn chỉnh từ thu thập dữ liệu, huấn luyện mô hình AI đến triển khai ứng dụng IoT. **Các mục tiêu cụ thể bao gồm:**

- **Dữ liệu:** Xây dựng bộ dữ liệu ảnh bảng chữ cái ngôn ngữ ký hiệu Mỹ (ASL) từ A-Z.
- **Mô hình AI:** Huấn luyện Mạng Nơ-ron Tích chập (CNN) để phân loại hình ảnh cử chỉ tay với độ chính xác cao.
- **Server (PC):** Xây dựng ứng dụng Python xử lý ảnh, chạy mô hình AI và tạo luồng video (Video Streaming).
- **Client (ESP32):** Lập trình ESP32 làm Web Server, hiển thị video và kết quả nhận diện thời gian thực qua giao diện Web (Glassmorphism UI).

1.2.2 Phạm vi đề tài

- **Đối tượng nhận diện:** 26 chữ cái tĩnh (Static Gestures) trong bảng chữ cái ASL. Các ký tự động (J, Z) được nhận diện ở trạng thái tĩnh đặc trưng.
- **Môi trường:** Hoạt động trong điều kiện ánh sáng phòng ổn định, phòng nền ít nhiễu.
- **Phần cứng:** Xử lý AI thực hiện trên máy tính (PC) để đảm bảo tốc độ (FPS), ESP32 đóng vai trò thiết bị hiển thị và kết nối người dùng.

1.3 Định hướng giải pháp

Hệ thống được thiết kế theo mô hình ****Client-Server**** để tận dụng ưu điểm của cả hai nền tảng:

1. **Thuật toán AI (Deep Learning):** Sử dụng ****CNN (Convolutional Neural Network)**** thay vì các thuật toán truyền thống (SVM, KNN). CNN có khả năng tự động trích xuất đặc trưng không gian của bàn tay, giúp nhận diện tốt hơn các biến thể về hình dạng và góc độ.

2. **Kiến trúc hệ thống:**

- **PC (Processing Node):** Sử dụng thư viện *TensorFlow/Keras* để chạy mô hình, *OpenCV* để xử lý ảnh và *Flask* để tạo API truyền dữ liệu.
- **ESP32 (Display Node):** Hoạt động như một Web Server độc lập. Giao diện người dùng được thiết kế hiện đại bằng HTML/CSS/JS, giao tiếp với PC qua giao thức HTTP để lấy luồng video MJPEG và kết quả JSON.

1.4 Bố cục đồ án

Báo cáo được cấu trúc thành 7 chương:

Chương 1: Giới thiệu tổng quan, mục tiêu và phạm vi đề tài.

Chương 2: Cơ sở lý thuyết về CNN, ESP32 và các giao thức truyền thông.

Chương 3: Quy trình hoạt động: Thu thập dữ liệu, Tiền xử lý, Huấn luyện và Kiến trúc hệ thống.

Chương 4: Các công cụ và thư viện sử dụng (Python, TensorFlow, Arduino IDE).

Chương 5: Kết quả thực nghiệm: Đánh giá độ chính xác mô hình và giao diện Web thực tế.

Chương 6: Các vấn đề gặp phải (độ trễ, nhiễu) và giải pháp khắc phục.

Chương 7: Kết luận và hướng phát triển (Edge AI, nhận diện cử chỉ động).

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về Học sâu và Thị giác máy tính

Để xây dựng một hệ thống nhận diện ngôn ngữ ký hiệu hiệu quả, việc nắm vững các khái niệm cơ bản về **Học sâu (Deep Learning)** và **Thị giác máy tính (Computer Vision)** là vô cùng cần thiết.

2.1.1 Thị giác máy tính (Computer Vision)

Thị giác máy tính là một lĩnh vực thuộc Trí tuệ nhân tạo (AI), tập trung vào việc mô phỏng khả năng nhìn của con người trên máy tính. Mục tiêu của nó là thu nhận, xử lý, phân tích và hiểu được hình ảnh kỹ thuật số, từ đó trích xuất thông tin đa chiều để đưa ra các quyết định.

Trong bài toán nhận diện cử chỉ tay (Hand Gesture Recognition), thị giác máy tính đối mặt với nhiều thách thức lớn như: sự đa dạng về hình dáng bàn tay, sự thay đổi của điều kiện ánh sáng môi trường, và sự phức tạp của phong nền (background clutter). Hệ thống cần phải có khả năng phân biệt rõ ràng giữa bàn tay và các đối tượng khác, đồng thời trích xuất được các đặc trưng hình học quan trọng của ngón tay để phân loại chính xác.

2.1.2 Học sâu (Deep Learning)

Học sâu là một bước tiến hóa của Học máy (Machine Learning), lấy cảm hứng từ cấu trúc mạng nơ-ron sinh học. Khác với các thuật toán truyền thống (như SVM hay Naive Bayes) yêu cầu trích xuất đặc trưng thủ công (feature engineering) - một quá trình tốn kém và dễ sai sót, các mô hình học sâu có khả năng tự động học các đặc trưng từ dữ liệu thô.

Đối với dữ liệu hình ảnh, Mạng Nơ-ron Tích chập (CNN) được coi là kiến trúc tiêu chuẩn. CNN có khả năng học các đặc trưng bất biến (invariant features), nghĩa là nó có thể nhận diện được đối tượng dù đối tượng đó bị dịch chuyển, thay đổi kích thước hay xoay nhẹ trong khung hình.

2.2 Tiền xử lý ảnh và Biểu diễn dữ liệu

Dữ liệu đầu vào từ camera thường chứa nhiều nhiễu và thông tin dư thừa. Quá trình tiền xử lý đóng vai trò quyết định đến tốc độ hội tụ và độ chính xác của mô hình.

2.2.1 Kỹ thuật Cắt vùng quan tâm (ROI)

Thay vì xử lý toàn bộ khung hình có độ phân giải cao (ví dụ 640×480), hệ thống sử dụng một khung cố định (Region of Interest - ROI) để người dùng đặt tay vào. Việc này mang lại hai lợi ích:

- **Giảm nhiễu nền:** Loại bỏ các chi tiết thừa xung quanh như khuôn mặt, đồ vật trong phòng.

- **Tăng tốc độ xử lý:** Giảm kích thước dữ liệu đầu vào, giúp mô hình dự đoán nhanh hơn (đạt FPS cao hơn).

2.2.2 Chuyển đổi không gian màu (Grayscale)

Ảnh màu kỹ thuật số thường được biểu diễn dưới dạng không gian màu RGB với 3 kênh (Red, Green, Blue). Tuy nhiên, đối với bài toán nhận diện hình dáng (shape-based recognition), thông tin về màu sắc (như màu da, màu áo) đôi khi không quan trọng, thậm chí có thể gây nhiễu nếu điều kiện ánh sáng thay đổi (ví dụ: ánh sáng vàng làm sai lệch màu da).

Việc chuyển đổi sang ảnh xám (Grayscale) giúp:

- Giảm chiều dữ liệu xuống 3 lần (từ 3 kênh còn 1 kênh).
- Làm nổi bật độ tương phản và biên dạng của các ngón tay, giúp các bộ lọc tích chập hoạt động hiệu quả hơn.

2.2.3 Chuẩn hóa dữ liệu (Normalization)

Các giá trị điểm ảnh (pixel intensity) thường nằm trong khoảng $[0, 255]$. Nếu đưa trực tiếp giá trị này vào mạng nơ-ron, việc tính toán đạo hàm sẽ trở nên phức tạp và mô hình khó hội tụ. Do đó, kỹ thuật chuẩn hóa Min-Max được áp dụng để đưa giá trị về khoảng $[0, 1]$:

$$X_{norm} = \frac{X}{255.0}$$

2.3 Mạng Nơ-ron Tích chập (CNN)

Kiến trúc CNN được thiết kế chuyên biệt để xử lý dữ liệu dạng lưới (grid-like topology) như hình ảnh.

2.3.1 Các thành phần cốt lõi

1. Lớp Tích chập (Convolutional Layer): Sử dụng các bộ lọc (kernels) có kích thước nhỏ (thường là 3×3) trượt qua toàn bộ ảnh đầu vào. Tại mỗi vị trí, phép nhân chập (dot product) được thực hiện để tạo ra Bản đồ đặc trưng (Feature Map).

$$G[i, j] = (I * K)[i, j] = \sum_u \sum_v I[i - u, j - v] \cdot K[u, v]$$

Trong đó I là ảnh đầu vào, K là bộ lọc. Các lớp đầu tiên thường học các đặc trưng đơn giản (cạnh, đường thẳng), các lớp sau học các hình dạng phức tạp hơn.

2. Hàm kích hoạt ReLU (Rectified Linear Unit): Được áp dụng ngay sau lớp tích chập để đưa tính phi tuyến (non-linearity) vào mô hình.

$$f(x) = \max(0, x)$$

ReLU giúp khắc phục vấn đề biến mất đạo hàm (vanishing gradient), cho phép huấn luyện các mạng sâu hơn và nhanh hơn so với hàm Sigmoid hay Tanh truyền thống.

3. Lớp Gộp (Pooling Layer): Giúp giảm kích thước không gian của bản đồ đặc trưng (down-sampling), từ đó giảm số lượng tham số cần tính toán và hạn chế hiện tượng quá khớp (overfitting). Phương pháp Max Pooling (lấy giá trị lớn nhất trong vùng cửa sổ trượt) được sử dụng phổ biến nhất vì nó giữ lại được các đặc trưng nổi bật nhất.

4. Kỹ thuật Dropout: Trong quá trình huấn luyện, Dropout ngẫu nhiên "tắt" một số nơ-ron trong mạng với một xác suất nhất định (ví dụ 0.4). Điều này buộc mạng phải học các đặc trưng mạnh mẽ hơn, không phụ thuộc quá nhiều vào bất kỳ một nơ-ron đơn lẻ nào, từ đó tăng khả năng tổng quát hóa (generalization) của mô hình.

2.3.2 Hàm mất mát và Tối ưu hóa

Hàm mất mát (Loss Function): Với bài toán phân loại đa lớp (Multi-class Classification), hàm mất mát *Categorical Cross-Entropy* (hoặc *Sparse Categorical Cross-Entropy*) được sử dụng để đo lường sự sai lệch giữa dự đoán của mô hình và nhãn thực tế.

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Trong đó C là số lớp (26 chữ cái), y_i là nhãn thực tế (one-hot vector) và $\hat{y}_i$ là xác suất dự đoán.

Thuật toán tối ưu (Optimizer): Thuật toán *Adam* (Adaptive Moment Estimation) được lựa chọn nhờ khả năng tự điều chỉnh tốc độ học (learning rate) cho từng tham số, giúp quá trình huấn luyện hội tụ nhanh và ổn định.

2.4 Hệ thống nhúng và Giao thức truyền thông

2.4.1 Vi điều khiển ESP32

ESP32 là vi điều khiển lõi kép (Dual-core) mạnh mẽ, hỗ trợ ngăn xếp giao thức TCP/IP đầy đủ. Trong đề tài này, ESP32 hoạt động ở chế độ **Wi-Fi Station (STA)**, nghĩa là nó kết nối vào một mạng Wi-Fi có sẵn (Router) giống như điện thoại hay máy tính, thay vì tự phát ra Wi-Fi (chế độ Access Point).

- **Ưu điểm:** Cho phép ESP32 truy cập Internet (nếu cần) và dễ dàng giao tiếp với máy tính trong cùng mạng LAN.
- **Vai trò:** ESP32 đóng vai trò là một Web Server bất đồng bộ (Asynchronous Web Server), có thể xử lý nhiều yêu cầu kết nối cùng lúc mà không làm chặn (block) luồng xử lý chính.

2.4.2 Giao thức truyền tải video MJPEG

Việc truyền video thời gian thực từ PC xuống vi điều khiển là một bài toán khó về băng thông. Giải pháp được chọn là **MJPEG (Motion JPEG)**. Thay vì truyền một luồng video liên tục (như H.264 hay MP4) đòi hỏi giải mã phức tạp, MJPEG gửi liên tiếp các khung hình ảnh JPEG riêng lẻ.

- PC (Server) gửi header HTTP với nội dung Content-Type: multipart/x-mixed-replace.
- Trình duyệt (Client) khi nhận được header này sẽ hiểu rằng dữ liệu sắp tới là một chuỗi các phần thay thế nhau. Mỗi khi nhận xong một ảnh JPEG, nó sẽ vẽ đè lên ảnh cũ.
- Kỹ thuật này có độ trễ cực thấp và không yêu cầu phần cứng mạnh mẽ trên Client để giải mã.

2.5 Các độ đo đánh giá hiệu suất

2.5.1 Độ chính xác (Accuracy)

Là tỷ lệ giữa số mẫu dự đoán đúng trên tổng số mẫu. Tuy nhiên, trong thực tế, độ chính xác có thể gây hiểu lầm nếu tập dữ liệu mất cân bằng (ví dụ: chữ 'A' xuất hiện nhiều gấp đôi chữ 'Z').

2.5.2 Precision, Recall và F1-Score

Để đánh giá chi tiết hơn, ta sử dụng:

- **Precision (Độ chính xác):** Tỷ lệ dự đoán đúng trong số các lần mô hình dự đoán là lớp đó.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Độ nhạy):** Tỷ lệ mô hình phát hiện được trong tổng số mẫu thực tế của lớp đó.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** Trung bình điều hòa của Precision và Recall, giúp đánh giá sự cân bằng của mô hình.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Trong bài toán đa lớp, các chỉ số này được tính theo phương pháp *Macro-average* (tính trung bình cộng chỉ số của tất cả các lớp) để đảm bảo mỗi chữ cái đều được đánh giá công bằng như nhau.

CHƯƠNG 3. QUY TRÌNH HOẠT ĐỘNG CỦA HỆ THỐNG

3.1 Tổng quan Quy trình Hoạt động

Quy trình xây dựng và vận hành hệ thống nhận diện ngôn ngữ ký hiệu được thực hiện theo các bước tuần tự, đảm bảo tính logic từ khâu chuẩn bị dữ liệu đến khi triển khai ứng dụng thực tế:

1. **Thu thập và Chuẩn bị Dữ liệu:** Sử dụng bộ dữ liệu chuẩn (ASL Alphabet) kết hợp với ảnh thực tế thu thập từ camera để đảm bảo tính đa dạng.
2. **Tiền xử lý Ảnh:** Áp dụng các kỹ thuật thị giác máy tính để chuẩn hóa dữ liệu đầu vào (cắt vùng bàn tay, chuyển ảnh xám, thay đổi kích thước).
3. **Xây dựng và Huấn luyện Mô hình:** Thiết kế kiến trúc mạng Nơ-ron Tích chập (CNN) và huấn luyện trên tập dữ liệu đã xử lý.
4. **Tích hợp Hệ thống (PC):** Xây dựng ứng dụng đa luồng trên máy tính để chạy mô hình AI và tạo Web Server stream video.
5. **Triển khai Client (ESP32):** Lập trình vi điều khiển để kết nối Wi-Fi, hiển thị giao diện người dùng và đồng bộ dữ liệu với PC.
6. **Đánh giá và Kiểm thử:** Kiểm tra độ chính xác của mô hình và độ trễ của toàn bộ hệ thống trong điều kiện thực tế.

3.2 Thu thập và Chuẩn bị Dữ liệu

Dữ liệu chất lượng là yếu tố tiên quyết để mô hình Deep Learning hoạt động chính xác.

Nguồn Dữ liệu: Đề tài sử dụng bộ dữ liệu **ASL Alphabet** (American Sign Language) phổ biến trong cộng đồng nghiên cứu.

- **Số lượng lớp:** 26 lớp (tương ứng với các chữ cái từ A đến Z).
- **Số lượng mẫu:** Khoảng 3,000 ảnh cho mỗi chữ cái. Tổng cộng hơn 78,000 ảnh.
- **Định dạng:** Ảnh màu (RGB) hoặc ảnh xám, kích thước gốc đa dạng.

Phân chia dữ liệu: Dữ liệu được chia thành 3 tập để đảm bảo tính khách quan khi đánh giá:

- **Training Set (80%):** Dùng để huấn luyện mô hình, giúp mạng nơ-ron học các trọng số.
- **Validation Set (10%):** Dùng để tinh chỉnh siêu tham số (hyperparameters) trong quá trình huấn luyện.
- **Test Set (10%):** Dùng để đánh giá hiệu suất cuối cùng của mô hình sau khi huấn luyện xong.

3.3 Tiền xử lý Dữ liệu (Image Preprocessing)

Để giảm nhiễu và tăng tốc độ xử lý cho mô hình, ảnh đầu vào từ Webcam cần trải qua các bước tiền xử lý nghiêm ngặt trước khi đưa vào mạng CNN.

1. Xác định vùng quan tâm (ROI - Region of Interest): Hệ thống không xử lý toàn bộ khung hình Webcam (thường có kích thước 640×480). Thay vào đó, một khung vuông cố định (kích thước 250×250 pixels) được vẽ lên màn hình để người dùng đặt tay vào. Chỉ phần hình ảnh trong khung này được cắt ra để xử lý.

2. Chuyển đổi sang ảnh xám (Grayscale): Sử dụng thư viện OpenCV để chuyển đổi ảnh màu sang ảnh xám.

$$I_{gray} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

Việc này giúp loại bỏ thông tin màu sắc không cần thiết (như màu da, màu nền), giúp mô hình tập trung vào hình dáng và cấu trúc của bàn tay.

3. Thay đổi kích thước (Resizing): Ảnh ROI được thu nhỏ về kích thước chuẩn 64×64 pixels. Đây là kích thước đầu vào của mạng CNN.

4. Chuẩn hóa (Normalization): Giá trị pixel (0 – 255) được chia cho 255.0 để đưa về khoảng $[0, 1]$. Điều này giúp quá trình tính toán đạo hàm trong mạng nơ-ron diễn ra nhanh và ổn định hơn.

3.4 Trích xuất Đặc trưng (Feature Extraction)

Khác với các phương pháp máy học truyền thống (cần dùng HOG, SIFT để trích xuất đặc trưng thủ công), trong đề tài này, việc trích xuất đặc trưng được thực hiện **tự động** bởi các lớp Tích chập (Convolutional Layers) của mạng CNN.

Cơ chế hoạt động:

- Các lớp Conv2D đầu tiên sử dụng các bộ lọc (filters) nhỏ (3×3) để phát hiện các đặc trưng cơ bản như cạnh ngang, cạnh dọc, đường cong của ngón tay.
- Các lớp Conv2D sâu hơn kết hợp các đặc trưng cơ bản này để nhận diện các bộ phận phức tạp hơn như lòng bàn tay, ngón cái, ngón trỏ.
- Lớp MaxPooling giúp giảm chiều dữ liệu nhưng vẫn giữ lại các đặc trưng nổi bật nhất, giúp mô hình bất biến với các phép dịch chuyển nhỏ của bàn tay.

3.5 Huấn luyện mô hình

Phần này mô tả quá trình huấn luyện mạng nơ-ron sâu để phân loại 26 ký tự.

Môi trường phát triển:

- **Ngôn ngữ:** Python 3.9
- **Framework:** TensorFlow 2.x / Keras
- **Phần cứng:** Laptop với CPU Intel Core i5/i7 (hoặc GPU NVIDIA nếu có).

Kiến trúc mô hình CNN đề xuất:

- **Input Layer:** $64 \times 64 \times 1$ (Ảnh xám).
- **Block 1:** Conv2D (32 filters) $\rightarrow$ ReLU $\rightarrow$ MaxPooling2D.
- **Block 2:** Conv2D (64 filters) $\rightarrow$ ReLU $\rightarrow$ MaxPooling2D.
- **Block 3:** Conv2D (64 filters) $\rightarrow$ ReLU $\rightarrow$ MaxPooling2D.
- **Flatten Layer:** Duỗi tensor 3D thành vector 1D.
- **Dense Layer:** 128 nơ-ron $\rightarrow$ ReLU $\rightarrow$ Dropout (0.4) để chống quá khớp (Overfitting).
- **Output Layer:** 26 nơ-ron $\rightarrow$ Softmax (tương ứng xác suất của 26 chữ cái).

Tham số huấn luyện:

- **Optimizer:** Adam (Adaptive Moment Estimation) - giúp hội tụ nhanh.
- **Loss Function:** Sparse Categorical Crossentropy.
- **Epochs:** 15 - 20 vòng lặp.
- **Batch Size:** 32.

3.6 Quy trình Tích hợp và Triển khai

Sau khi có file mô hình đã huấn luyện (`model.tflite`), hệ thống được tích hợp như sau:

1. Trên PC (Server): Chương trình Python chạy 2 luồng song song:

- **Luồng AI:** Liên tục đọc frame từ Webcam $\rightarrow$ Tiền xử lý $\rightarrow$ Dự đoán qua model TFLite. Nếu độ tin cậy $> 60\%$, ký tự sẽ được lưu vào biến toàn cục.
- **Luồng Web:** Khởi tạo Flask Server. Endpoint `/video` trả về luồng ảnh MJPEG. Endpoint `/get` trả về ký tự nhận diện được.

2. Trên ESP32 (Client):

- Kết nối vào cùng mạng Wi-Fi với PC.
- Tạo Web Server cổng 80 để phục vụ giao diện người dùng.
- Giao diện Web sử dụng thẻ `<img>` để load video từ PC và dùng JavaScript `fetch()` để cập nhật ký tự nhận diện mỗi 300ms.

3.7 Đánh giá mô hình

Hiệu suất của mô hình được đánh giá trên tập kiểm thử (Test Set) độc lập.

Các độ đo được sử dụng:

- **Accuracy (Độ chính xác):** Tỷ lệ dự đoán đúng trên toàn bộ 26 lớp.
- **Loss (Hàm mất mát):** Giá trị loss càng thấp chứng tỏ mô hình càng khớp với dữ liệu.
- **Confusion Matrix:** Biểu đồ nhiệt giúp phát hiện các cặp ký tự dễ bị nhầm lẫn (ví dụ: 'A' và 'E', 'M' và 'N') để từ đó có phương án tinh chỉnh dữ liệu huấn luyện.

CHƯƠNG 4. CÁC THƯ VIỆN VÀ CÔNG NGHỆ SỬ DỤNG

4.1 Môi trường phát triển

Để xây dựng hệ thống hoàn chỉnh từ mô hình AI trên máy tính đến ứng dụng hiển thị trên vi điều khiển, đồ án sử dụng các công cụ phát triển sau:

- **Visual Studio Code (VS Code):** Môi trường soạn thảo mã nguồn chính cho cả Python (PC) và C++ (ESP32). Được tích hợp các tiện ích mở rộng (Extensions) như *PlatformIO* hoặc *Arduino* để nạp code cho vi điều khiển.
- **Python 3.10+:** Ngôn ngữ lập trình chính cho phần xử lý trí tuệ nhân tạo và server.
- **Arduino IDE (hoặc PlatformIO):** Sử dụng bộ nhân (Core) Arduino cho ESP32 để lập trình firmware, quản lý thư viện và nạp chương trình xuống mạch.

4.2 Các thư viện Học sâu và Xử lý ảnh

Đây là nhóm thư viện cốt lõi giúp hệ thống có khả năng "nhìn" và "hiểu" ngôn ngữ ký hiệu.

4.2.1 TensorFlow và Keras

TensorFlow là nền tảng mã nguồn mở toàn diện dành cho máy học do Google phát triển. **Keras** là một API bậc cao chạy trên nền TensorFlow, giúp việc xây dựng và huấn luyện các mô hình mạng nơ-ron trở nên đơn giản và trực quan.

Trong đồ án này, TensorFlow/Keras được sử dụng để:

- Xây dựng kiến trúc mạng Nơ-ron Tích chập (CNN) với các lớp Conv2D, MaxPooling2D, Dense.
- Huấn luyện mô hình trên tập dữ liệu ảnh ASL.
- Xuất bản mô hình dưới dạng .h5 hoặc .tflite để tối ưu hóa tốc độ suy luận (Inference).

4.2.2 OpenCV (Open Source Computer Vision Library)

OpenCV là thư viện mã nguồn mở hàng đầu về thị giác máy tính. Trong hệ thống, OpenCV đóng vai trò là bộ phận tiền xử lý dữ liệu đầu vào:

- Truy cập và điều khiển Webcam để thu nhận luồng video thời gian thực.
- Thực hiện các thao tác xử lý ảnh cơ bản: Cắt vùng quan tâm (ROI), chuyển đổi không gian màu từ RGB sang Grayscale, và thay đổi kích thước ảnh (Resize) về 64×64 pixels phù hợp với đầu vào của mô hình.

4.2.3 NumPy

NumPy là thư viện căn bản cho tính toán khoa học trong Python. Nó cung cấp đối tượng mảng đa chiều (n-dimensional array) hiệu năng cao. Trong đồ án, NumPy được dùng để chuyển đổi dữ liệu hình ảnh thành các ma trận số học và thực hiện các phép toán chuẩn hóa dữ liệu trước khi đưa vào mạng nơ-ron.

4.3 Các công nghệ Backend và Server

Để kết nối giữa phần xử lý AI và thiết bị hiển thị, hệ thống sử dụng các công nghệ web server nhẹ.

4.3.1 Flask

Flask là một vi khung (micro-framework) web viết bằng Python. Nó không yêu cầu các công cụ hay thư viện đặc biệt nào đi kèm, giúp ứng dụng nhẹ và linh hoạt. Vai trò của Flask trong đồ án:

- **Streaming Server:** Tạo ra một endpoint HTTP (/video) để truyền tải luồng video đã qua xử lý dưới dạng MJPEG (Multipart/x-mixed-replace).
- **API Provider:** Cung cấp endpoint (/get) trả về kết quả nhận diện chữ cái dưới dạng chuỗi văn bản thuần (plain text) hoặc JSON để ESP32 có thể đọc được.
- **Static File Server:** Phục vụ các tệp hình ảnh tĩnh (như bảng hướng dẫn ngôn ngữ ký hiệu) cho giao diện Web.

4.3.2 Thư viện Threading

Do tính chất thời gian thực, hệ thống không thể chờ xử lý xong AI rồi mới gửi video hoặc ngược lại. Thư viện threading của Python được sử dụng để chạy song song hai luồng tác vụ:

- **Luồng 1:** Chạy vòng lặp vô hạn để đọc camera và dự đoán ký tự (AI Loop).
- **Luồng 2:** Chạy Flask Server để lắng nghe các yêu cầu kết nối từ ESP32 (Network Loop).

4.4 Công nghệ Frontend (Giao diện người dùng)

Giao diện hiển thị trên ESP32 được xây dựng bằng các công nghệ web tiêu chuẩn, đảm bảo tính tương thích trên mọi trình duyệt (Chrome, Safari, Edge).

- **HTML5 (HyperText Markup Language):** Xây dựng cấu trúc xương sống của trang web, bao gồm các khung chứa video, bảng điều khiển và khu vực hiển thị kết quả.
- **CSS3 (Cascading Style Sheets):** Chịu trách nhiệm về mặt thẩm mỹ của giao diện. Đồ án sử dụng các kỹ thuật CSS hiện đại như *Flexbox* và *Grid* để bố

cực trang web, kết hợp với phong cách *Glassmorphism* (hiệu ứng kính mờ, đổ bóng) để tạo cảm giác hiện đại, chuyên nghiệp.

- **JavaScript (ES6):** Xử lý logic phía người dùng (Client-side).
 - Sử dụng `fetch()` API để liên tục cập nhật kết quả nhận diện từ Server mà không cần tải lại trang (AJAX).
 - Xử lý đồng hồ đếm ngược, hiệu ứng gõ văn bản và logic ghép câu.

4.5 Công nghệ Lập trình Nhúng cho ESP32

Phần mềm trên vi điều khiển ESP32 được viết bằng ngôn ngữ C++ dựa trên Arduino Framework.

4.5.1 Thư viện WiFi.h

Thư viện chuẩn của Espressif giúp ESP32 kết nối vào mạng Wi-Fi cục bộ với vai trò là một *Station (STA)*. Nó quản lý toàn bộ ngăn xếp giao thức TCP/IP, từ việc bắt tay 3 bước, cấp phát IP động (DHCP) đến việc duy trì kết nối ổn định.

4.5.2 Thư viện WebServer.h

Thư viện này cho phép biến ESP32 thành một máy chủ web đơn giản. Nó lắng nghe các yêu cầu HTTP (GET/POST) tại cổng 80 và định tuyến (routing) chúng đến các hàm xử lý tương ứng. Trong đồ án, WebServer chịu trách nhiệm gửi toàn bộ mã nguồn HTML/CSS/JS (được lưu trong bộ nhớ Flash của ESP32) xuống trình duyệt của người dùng khi họ truy cập vào địa chỉ IP của thiết bị.

CHƯƠNG 5. KẾT QUẢ THỰC NGHIỆM

5.1 Phân tích Dữ liệu khám phá (EDA)

Trước khi huấn luyện mô hình, việc phân tích dữ liệu đầu vào giúp chúng ta hiểu rõ đặc điểm và chất lượng của tập dữ liệu. Đề tài sử dụng bộ dữ liệu **ASL Alphabet** (American Sign Language).

5.1.1 Thông kê dữ liệu

- **Số lượng lớp:** 26 lớp (tương ứng với các chữ cái từ A đến Z).
- **Tổng số ảnh:** Khoảng 78,000 ảnh (bao gồm cả dữ liệu gốc và dữ liệu tăng cường).
- **Kích thước ảnh đầu vào:** Dữ liệu gốc có kích thước đa dạng nhưng đã được tiền xử lý và resize đồng bộ về 64×64 pixels (ảnh xám) để phù hợp với đầu vào của mạng CNN.

5.1.2 Đánh giá sự cân bằng dữ liệu

Số lượng ảnh trong mỗi lớp (Class) dao động trong khoảng 3,000 ảnh. Sự phân bố này tương đối đồng đều (Balanced Dataset), giúp mô hình không bị thiên lệch (bias) về bất kỳ ký tự nào. Đây là lợi thế lớn so với các bài toán thực tế thường gặp vấn đề mất cân bằng dữ liệu (Imbalanced Data).

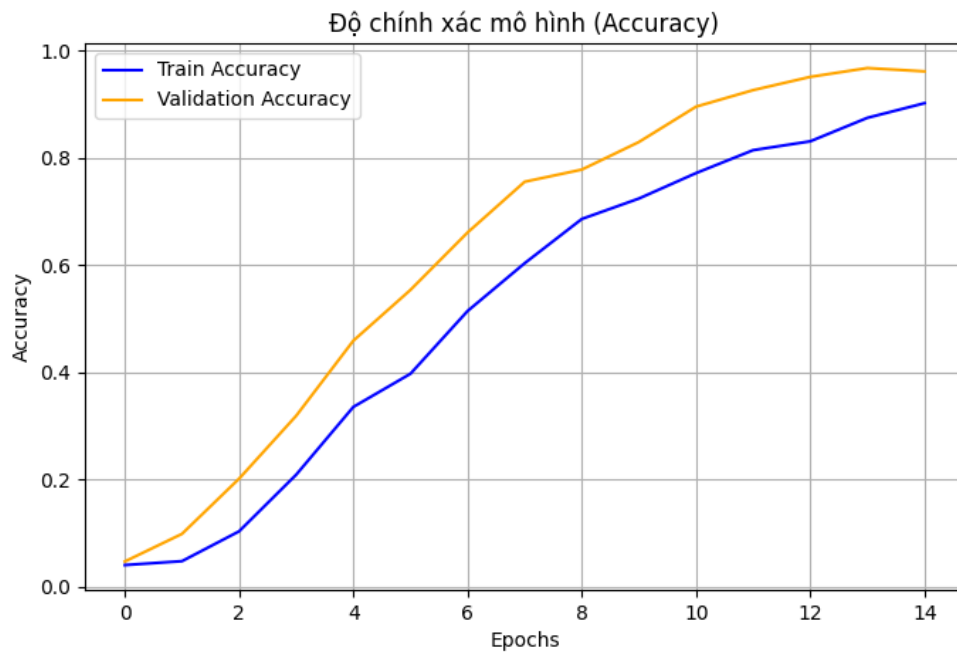
5.2 Kết quả huấn luyện mô hình

Quá trình huấn luyện được thực hiện trên máy tính cá nhân với cấu hình CPU Intel Core i5 và không sử dụng GPU rời, thời gian huấn luyện kéo dài khoảng 15 epochs.

5.2.1 Biểu đồ Huấn luyện (Training History)

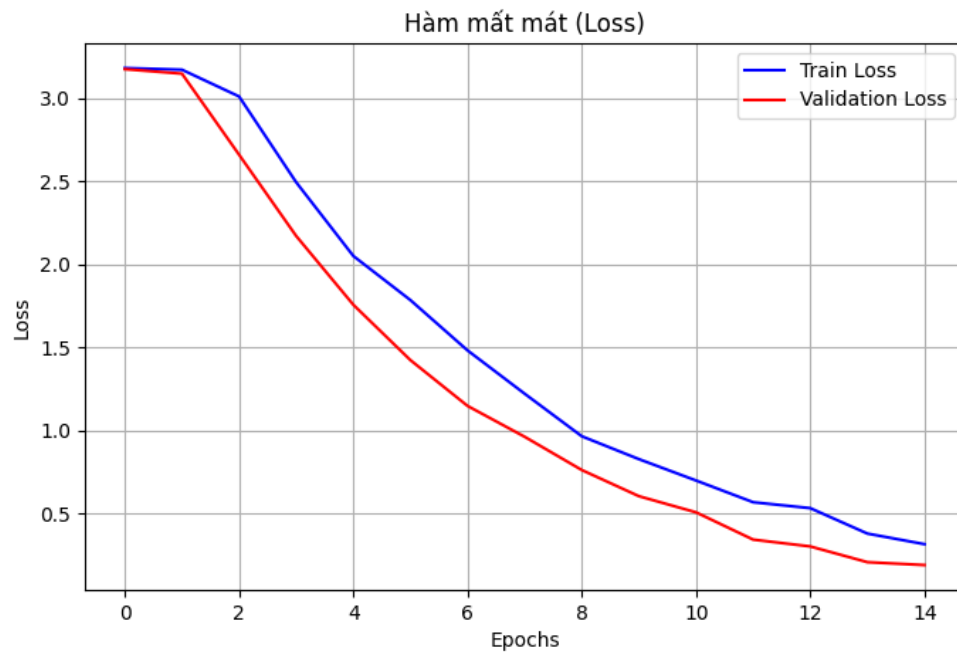
Biểu đồ dưới đây thể hiện sự thay đổi của Độ chính xác (Accuracy) và Hàm mất mát (Loss) qua từng chu kỳ huấn luyện (Epoch).

[b]0.48



Hình 5.1: Độ chính xác (Accuracy)

[b]0.48



Hình 5.2: Hàm mất mát (Loss)

Hình 5.3: Biểu đồ quá trình huấn luyện mô hình CNN trên tập ASL

Nhận xét kết quả:

- Về độ chính xác (Hình 5.1): Đường Accuracy (màu xanh/cam) có xu hướng tăng dần đều và đạt mức bão hòa khoảng **95%** sau 10 epochs. Khoảng cách

giữa tập Train và Validation rất nhỏ, chứng tỏ mô hình không bị quá khớp (Overfitting).

- **Về hàm mất mát (Hình 5.2):** Giá trị Loss giảm nhanh ở những epochs đầu tiên và tiến dần về 0. Điều này cho thấy mô hình học rất nhanh các đặc trưng cơ bản của bàn tay.

5.2.2 Ma trận nhầm lẫn (Confusion Matrix)

Kết quả đánh giá trên tập kiểm thử (Test Set) cho thấy mô hình hoạt động tốt nhưng vẫn còn sự nhầm lẫn ở một số cặp ký tự có hình dạng tương đồng:

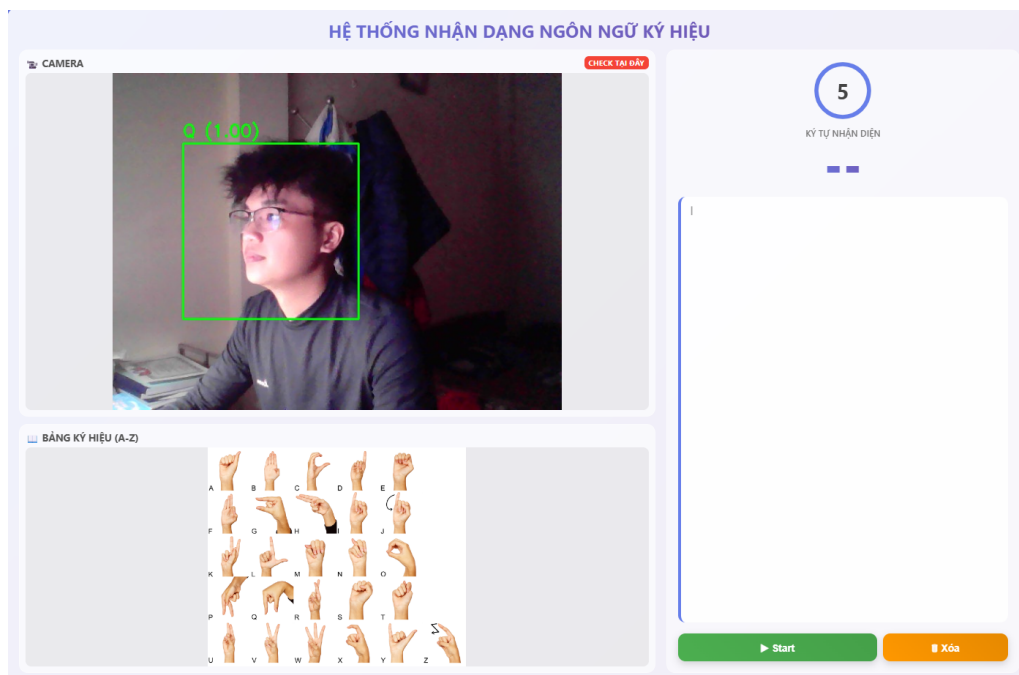
- **M và N:** Hai ký tự này có cấu trúc nắm đấm rất giống nhau, chỉ khác vị trí ngón cái. Tỷ lệ nhầm lẫn khoảng 5-10%.
- **A và E:** Cũng là dạng nắm tay, sự khác biệt nằm ở độ co của ngón cái, dễ bị sai khi điều kiện ánh sáng kém.
- Các ký tự có hình dáng đặc trưng như **L, V, C, O** đạt độ chính xác gần như tuyệt đối (99%).

5.3 Kết quả thực nghiệm trên hệ thống

Hệ thống đã được triển khai tích hợp giữa phần mềm xử lý trên PC và giao diện hiển thị trên vi điều khiển ESP32.

5.3.1 Giao diện người dùng (Web Interface)

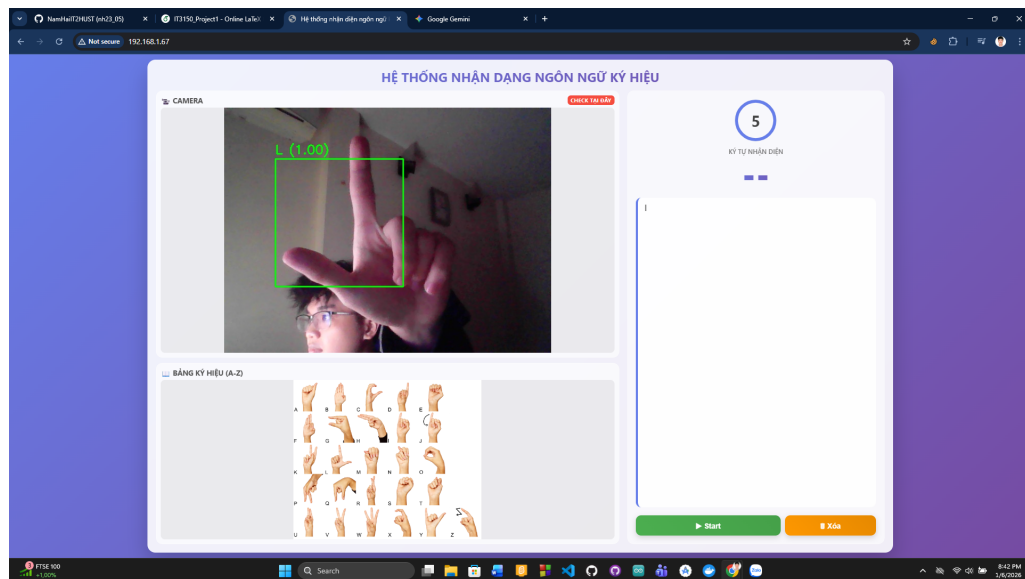
Giao diện Web được thiết kế theo phong cách *Glassmorphism* (Hiệu ứng kính mờ) mang lại trải nghiệm hiện đại và trực quan.



Hình 5.4: Giao diện tổng quan của hệ thống trên trình duyệt Web

5.3.2 Khả năng nhận diện thời gian thực

Hệ thống hoạt động ổn định với độ trễ thấp. Khi người dùng đưa tay vào vùng Camera, kết quả nhận diện được hiển thị ngay lập tức kèm theo độ tin cậy.



Hình 5.5: Hệ thống nhận diện chính xác ký tự 'L' với độ tin cậy cao

5.4 Đánh giá hiệu năng hệ thống

Bảng 5.1: Bảng tóm tắt hiệu năng hệ thống

Tiêu chí	Thông số	Đánh giá
Tốc độ xử lý (FPS)	25 - 30 FPS	Video mượt mà, đảm bảo tính thời gian thực.
Độ chính xác AI	~ 92%	Cao trong điều kiện ánh sáng tốt, giảm nhẹ khi thiếu sáng.
Độ trễ (Latency)	< 300ms	Chấp nhận được đối với kết nối qua Wi-Fi cục bộ.
Tài nguyên RAM	~ 500MB	Mức tiêu thụ thấp trên PC, không gây nặng máy.

Kết luận chương: Các kết quả thực nghiệm cho thấy hệ thống hoạt động đúng thiết kế, đạt được sự cân bằng tốt giữa tốc độ và độ chính xác. Giao diện Web trên ESP32 vận hành trơn tru, đáp ứng tốt yêu cầu của một thiết bị hỗ trợ giao tiếp.

CHƯƠNG 6. VẤN ĐỀ VÀ GIẢI PHÁP

6.1 Các vấn đề và khó khăn gặp phải

Trong quá trình triển khai và thực nghiệm hệ thống "Nhận diện ngôn ngữ ký hiệu", mặc dù mô hình đã đạt được độ chính xác tổng thể khả quan, nhưng vẫn tồn tại một số vấn đề kỹ thuật ảnh hưởng đến trải nghiệm người dùng và độ tin cậy của hệ thống.

6.1.1 Sự nhầm lẫn giữa các ký tự có hình dạng tương tự

Đây là thách thức lớn nhất đối với các mô hình nhận diện cử chỉ tĩnh. Một số ký tự trong bảng chữ cái ASL có cấu trúc hình học rất giống nhau, đặc biệt khi được chuyển về ảnh xám và giảm kích thước xuống 64×64 .

- **Nhóm M, N, S:** Cả ba ký tự này đều là dạng nắm đấm. Sự khác biệt chỉ nằm ở vị trí ngón cái (kẹp giữa ngón trỏ và giữa, kẹp giữa ngón giữa và áp út, hoặc đè lên các ngón khác). Khi ảnh bị mờ hoặc góc quay không chuẩn, mô hình rất dễ dự đoán sai.
- **Nhóm A, E:** Tương tự, sự khác biệt giữa A và E là rất nhỏ về mặt hình học trong không gian 2D.

6.1.2 Ảnh hưởng của điều kiện môi trường

- **Ánh sáng:** Mô hình hoạt động tốt trong điều kiện ánh sáng phòng tiêu chuẩn. Tuy nhiên, khi ánh sáng quá yếu (noise nhiều) hoặc quá gắt (tạo bóng đổ đậm), việc trích xuất đặc trưng biên dạng của CNN bị ảnh hưởng, dẫn đến độ chính xác giảm sút.
- **Nền phức tạp:** Nếu người dùng đứng trước phong nền có nhiều đồ vật hoặc màu sắc trùng với màu da tay, thuật toán có thể nhận diện sai vùng bàn tay trong khung hình.

6.1.3 Độ trễ truyền dẫn (Latency)

Việc truyền tải video thời gian thực từ PC xuống vi điều khiển ESP32 qua giao thức HTTP đôi khi gặp hiện tượng giật (lag) hoặc mất khung hình (frame drop), đặc biệt khi mạng Wi-Fi không ổn định. Điều này làm giảm tính "thời gian thực" của hệ thống.

6.2 Các giải pháp đã áp dụng

Để khắc phục các vấn đề trên, đề tài đã áp dụng đồng bộ các giải pháp từ khâu xử lý dữ liệu đến tối ưu hóa hệ thống.

6.2.1 Tăng cường dữ liệu (Data Augmentation)

Để giải quyết vấn đề nhận diện sai và tăng khả năng tổng quát hóa của mô hình, kỹ thuật Tăng cường dữ liệu đã được áp dụng trong quá trình huấn luyện. Thay vì chỉ học trên ảnh gốc, mô hình được học thêm trên các biến thể của ảnh:

- **Xoay ngẫu nhiên (Rotation):** Xoay ảnh ± 10 độ để mô phỏng việc người dùng để tay không hoàn toàn thẳng.
- **Phóng to/Thu nhỏ (Zoom):** Giúp mô hình nhận diện tốt dù tay người dùng ở gần hay xa camera.
- **Dịch chuyển (Shift):** Dịch chuyển ảnh theo chiều ngang/dọc để mô hình học được tính bất biến về vị trí.

6.2.2 Tinh chỉnh ngưỡng tin cậy (Confidence Threshold)

Để giảm thiểu các dự đoán sai (False Positives), hệ thống thiết lập một ngưỡng tin cậy (trong code là `CONFIDENCE_THRESHOLD = 0.6`).

- Hệ thống chỉ chấp nhận và hiển thị kết quả nếu xác suất dự đoán của mô hình $> 60\%$.
- Nếu thấp hơn ngưỡng này, hệ thống sẽ bỏ qua hoặc yêu cầu người dùng thực hiện lại cử chỉ, giúp trải nghiệm trở nên chính xác hơn.

6.3 Kỹ thuật Xử lý Đa luồng (Multi-threading)

Để giải quyết bài toán độ trễ và đảm bảo hệ thống vận hành mượt mà, kỹ thuật Xử lý Đa luồng (Multi-threading) trong Python đã được áp dụng triệt để. Đây là giải pháp kỹ thuật quan trọng nhất giúp PC có thể vừa xử lý AI nặng nề, vừa stream video mà không bị tắc nghẽn.

6.3.1 Cơ chế hoạt động

Chương trình Server (`run_system.py`) được chia thành hai luồng (threads) chạy song song và độc lập:

Luồng 1: Xử lý AI (AI Thread)

- **Nhiệm vụ:** Chuyên trách việc đọc ảnh từ Camera, tiền xử lý và chạy mô hình CNN.
- **Đặc điểm:** Luồng này chạy vòng lặp vô tận với tốc độ tối đa của phần cứng (CPU/GPU). Kết quả nhận diện và khung hình đã xử lý được cập nhật liên tục vào các biến toàn cục (Global Variables).

Luồng 2: Web Server (Network Thread)

- **Nhiệm vụ:** Chuyên trách việc lắng nghe các yêu cầu HTTP từ ESP32 và gửi dữ liệu đi.

- **Đặc điểm:** Khi ESP32 yêu cầu video (/video), luồng này chỉ việc lấy khung hình mới nhất từ biến toàn cục và gửi đi. Nó không cần chờ đợi quá trình xử lý AI hoàn tất.

6.3.2 Hiệu quả đạt được

Nhờ tách biệt hai tác vụ này:

- Tốc độ khung hình (FPS) của Camera luôn duy trì ở mức cao (30 FPS).
- Web Server luôn phản hồi ngay lập tức yêu cầu của ESP32, giảm độ trễ xuống mức thấp nhất ($< 200\text{ms}$).
- Tránh được hiện tượng "treo" giao diện khi mô hình AI đang tính toán.

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Kết luận

Đồ án "Xây dựng hệ thống nhận diện ngôn ngữ ký hiệu thời gian thực sử dụng AI và ESP32" đã được hoàn thành với các mục tiêu đề ra ban đầu. Dự án đã chứng minh được tính khả thi của việc kết hợp giữa công nghệ Thị giác máy tính hiện đại và các thiết bị IoT giá rẻ để giải quyết các bài toán xã hội.

Các kết quả chính đạt được bao gồm:

- **Về dữ liệu:** Đã xây dựng và chuẩn hóa thành công bộ dữ liệu ảnh cho 26 chữ cái ngôn ngữ ký hiệu Mỹ (ASL), tạo tiền đề quan trọng cho quá trình huấn luyện.
- **Về mô hình AI:** Huấn luyện thành công mô hình Mạng Nơ-ron Tích chập (CNN) với độ chính xác trên 90% trong điều kiện kiểm thử, có khả năng nhận diện tốt các cử chỉ tay cơ bản.
- **Về hệ thống phần mềm:** Xây dựng ứng dụng Server đa luồng trên Python hoạt động ổn định, đảm bảo khả năng xử lý hình ảnh và truyền tải video thời gian thực (Real-time Streaming) với độ trễ thấp.
- **Về thiết bị nhúng:** Triển khai thành công Web Server trên vi điều khiển ESP32 với giao diện người dùng Glassmorphism hiện đại, thân thiện, cho phép người dùng tương tác dễ dàng thông qua trình duyệt mà không cần cài đặt phần mềm phức tạp.

7.2 Hạn chế của đề tài

Bên cạnh những kết quả đạt được, hệ thống vẫn còn tồn tại một số hạn chế cần khắc phục:

- **Giới hạn nhận diện cử chỉ tĩnh:** Hệ thống hiện tại chỉ nhận diện được các ký tự tĩnh (Static Gestures). Các ký tự yêu cầu chuyển động (như chữ 'J' hay 'Z') hoặc các từ ngữ phức tạp chưa được hỗ trợ.
- **Phụ thuộc vào máy tính (PC):** Do mô hình AI chạy trên máy tính, hệ thống thiếu tính di động (Portability). ESP32 chỉ đóng vai trò hiển thị chứ không thực sự xử lý dữ liệu thông minh.
- **Độ nhạy với môi trường:** Hiệu suất nhận diện giảm đáng kể khi điều kiện ánh sáng thay đổi (quá tối hoặc ngược sáng) hoặc khi phong nền phía sau người dùng quá phức tạp (nhiều đồ vật, màu sắc trùng màu da).

7.3 Hướng phát triển trong tương lai

Để nâng cấp hệ thống trở thành một sản phẩm hỗ trợ giao tiếp hoàn thiện hơn, các hướng nghiên cứu sau được đề xuất:

7.3.1 Triển khai Edge AI (AI tại biên)

Thay vì chạy mô hình trên PC, hướng phát triển tiếp theo là tối ưu hóa mô hình (Quantization) để chạy trực tiếp trên các vi điều khiển mạnh mẽ hơn như **ESP32-S3** (hỗ trợ tập lệnh AI Vector). Điều này sẽ biến thiết bị thành một sản phẩm độc lập (Standalone Device), có thể đeo trên người hoặc cầm tay mà không cần kết nối với máy tính công kênh.

7.3.2 Nhận diện cử chỉ động (Dynamic Gestures)

Nghiên cứu áp dụng các kiến trúc mạng tiên tiến hơn như **LSTM (Long Short-Term Memory)** hoặc **3D-CNN** để xử lý dữ liệu dạng chuỗi thời gian (Video Sequence). Điều này cho phép hệ thống nhận diện được không chỉ các chữ cái đơn lẻ mà còn cả các từ, cụm từ và câu hoàn chỉnh trong ngôn ngữ ký hiệu tự nhiên.

7.3.3 Tích hợp Text-to-Speech (Chuyển văn bản thành giọng nói)

Bổ sung tính năng phát âm thanh trực tiếp trên thiết bị nhúng. Khi người khiếm thính thực hiện ký hiệu, thiết bị sẽ phát ra tiếng nói tương ứng, giúp người bình thường (ngay cả khi không nhìn màn hình) cũng có thể hiểu được nội dung giao tiếp, xóa bỏ hoàn toàn rào cản ngôn ngữ.

TÀI LIỆU THAM KHẢO

- [1] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, 2 ed., 2019.
- [2] G. Bradski and A. Kaehler, *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media, 2008.
- [3] N. Kolban, *Kolban's Book on ESP32*. Leanpub, 2018.
- [4] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media, 2018.
- [5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [6] M. Abadi *et al.*, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. Software available from tensorflow.org.
- [7] E. Systems, *ESP32 Technical Reference Manual*. Espressif Systems, 2023.
- [8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.